Backlog de Produto — Sistema Biblioteca no Kubernetes

Visão do produto

Disponibilizar o Sistema Biblioteca de forma confiável em Kubernetes, com frontend e API reunidos em uma única imagem de aplicação. O produto deve preservar as funções de cadastro de livros e empréstimos, publicar versões no GHCR e permitir escala, recuperação de falhas e retorno a uma versão estável quando necessário.

Objetivo deste backlog

Este backlog organiza os requisitos das Atividades Integradoras 1 e 2 antes do planejamento das sprints. Cada item descreve uma necessidade de produto e sua condição de aceite. Os itens estão ordenados por prioridade e ainda não possuem responsável, data ou status de sprint.

Critério de prioridade

- **P0 – Essencial:** necessário para entregar e demonstrar o sistema no Kubernetes.
- **P1 – Importante:** necessário para completar a demonstração de operação e manutenção.
- **P2 – Evolução:** melhora a rastreabilidade, a segurança ou a experiência de operação.

Premissas de arquitetura

- Frontend e API devem ser entregues juntos em um único container de aplicação.
- O PostgreSQL continua separado como serviço interno de dados.
- O PostgreSQL não deve ficar exposto publicamente.
- O frontend e a API devem manter comunicação configurável por variáveis de ambiente.
- As imagens devem ser versionadas no GitHub Container Registry (GHCR).
- Os manifests Kubernetes devem permanecer versionados no mesmo repositório.

Product Backlog priorizado

Ordem	ID	Épico	História de usuário	Prioridade	Critérios de aceite	Dependências
1	**PB-01**	**Aplicação em um único container**	Como usuário, quero abrir a interface e usar a API do Sistema Biblioteca a partir de uma única imagem de aplicação para simplificar a implantação e a demonstração.	**P0**	O frontend é compilado junto com a API; a API entrega os arquivos do frontend; apenas uma imagem da aplicação é gerada; navegar pela interface e chamar a API funciona no mesmo container.	Nenhuma
2	**PB-02**	**Arquitetura de implantação**	Como equipe, queremos definir como a aplicação unificada e o banco serão implantados no Kubernetes para que o sistema funcione como uma solução integrada.	**P0**	A aplicação possui um único Deployment; o banco possui responsabilidade separada; o banco permanece acessível apenas internamente.	PB-01

Ordem	ID	Épico	História de usuário	Prioridade	Critérios de aceite	Dependências
3	**PB-03**	**Configuração segura**	Como equipe, queremos externalizar URLs, origens permitidas e credenciais para evitar segredos no código e permitir ambientes diferentes.	**P0**	Arquivos de exemplo existem; segredos não entram no Git; CORS e conexão com banco são configuráveis sem alterar o código.	PB-02
4	**PB-04**	**Qualidade da aplicação**	Como equipe, queremos validar frontend e backend antes de publicar imagens para reduzir a chance de implantar uma versão com falha.	**P0**	Testes do frontend e backend executam; falhas interrompem a publicação; há evidências do resultado dos testes.	PB-01
5	**PB-05**	**Imagem Docker única**	Como equipe, queremos construir uma única imagem Docker do Sistema Biblioteca para executar a interface e a API de forma reproduzível.	**P0**	Existe um Dockerfile de aplicação unificada; o .dockerignore está no contexto correto; a imagem constrói sem erro; a interface e a API iniciam localmente no mesmo container.	PB-01, PB-03, PB-04
6	**PB-06**	**Banco de dados no cluster**	Como equipe, queremos disponibilizar o PostgreSQL de forma interna e persistente para preservar livros e empréstimos.	**P0**	Banco possui armazenamento persistente; credenciais chegam por Secret; a aplicação conecta ao banco por Service interno; porta do banco não é publicada para usuários externos.	PB-02, PB-03
7	**PB-07**	**Publicação no GHCR**	Como equipe, queremos publicar a imagem inicial unificada no GHCR para que o cluster use uma versão rastreável.	**P0**	A imagem `1.0.0` fica disponível; o workflow só publica após testes aprovados; a imagem pode ser baixada com `docker pull`.	PB-04, PB-05
8	**PB-08**	**Aplicação no Kubernetes**	Como bibliotecário, quero que o Sistema Biblioteca execute com duas réplicas para que o acesso continue disponível se um Pod falhar.	**P0**	Existe um Deployment da aplicação com 2 réplicas; readiness e liveness probes usam endpoint de saúde; dois Pods ficam `Running` e `Ready`.	PB-06, PB-07
9	**PB-09**	**Acesso estável**	Como usuário, quero acessar o Sistema Biblioteca por um Service estável para não depender dos endereços temporários dos Pods.	**P0**	O Service seleciona os Pods corretos; porta e targetPort correspondem ao container; a interface e a API respondem por navegador ou requisição HTTP.	PB-08
10	**PB-10**	**Implantação inicial**	Como equipe, queremos aplicar os manifests no cluster para disponibilizar a primeira versão operacional do Sistema Biblioteca.	**P0**	`kubectl apply` cria os recursos; o Deployment fica disponível; o Service é listado; a demonstração mostra cadastro, consulta ou empréstimo funcionando.	PB-08, PB-09
11	**PB-11**	**Observabilidade básica**	Como equipe, queremos consultar o estado dos Pods e Deployment para identificar rapidamente se a aplicação está pronta.	**P1**	Existe roteiro com `kubectl get pods`, `kubectl get deployments`, logs e descrição de eventos; a equipe consegue identificar Pod não pronto.	PB-10
12	**PB-12**	**Escalabilidade horizontal**	Como equipe, queremos aumentar a aplicação de 2 para 4 réplicas para atender a uma demanda maior.	**P1**	O Deployment chega a 4 Pods prontos; a criação dos novos Pods é registrada; o sistema continua acessível pelo Service.	PB-10
13	**PB-13**	**Ajuste de capacidade**	Como equipe, queremos reduzir a aplicação de 4 para 2 réplicas para adequar o uso de recursos quando a demanda diminuir.	**P1**	O cluster termina os Pods excedentes; duas réplicas permanecem prontas; o manifesto fica alinhado com a quantidade desejada.	PB-12
14	**PB-14**	**Recuperação automática**	Como equipe, queremos que o Kubernetes recrie um Pod removido para manter a disponibilidade configurada.	**P1**	Um Pod controlado pelo Deployment é removido; um novo Pod é criado automaticamente; duas réplicas voltam a ficar prontas.	PB-13
15	**PB-15**	**Nova versão do produto**	Como bibliotecário, quero receber uma melhoria pequena e visível para confirmar que o processo de atualização entrega valor.	**P1**	A mudança possui teste e documentação; o número de versão é atualizado; a alteração é revisável no histórico Git.	PB-04
16	**PB-16**	**Publicação da versão 1.0.1**	Como equipe, queremos publicar a nova imagem no GHCR sem remover a versão anterior para permitir uma atualização segura.	**P1**	A imagem `1.0.1` fica disponível; a imagem `1.0.0` continua disponível; tags e digests podem ser conferidos.	PB-15

Ordem	ID	Épico	História de usuário	Prioridade	Critérios de aceite	Dependências
17	**PB-17**	**Atualização controlada**	Como usuário, quero que o cluster atualize o Sistema Biblioteca para a nova imagem sem deixar o sistema indisponível.	**P1**	O Deployment usa a imagem `1.0.1`; o rollout termina com todas as réplicas prontas; a funcionalidade alterada pode ser demonstrada.	PB-16
18	**PB-18**	**Rollback**	Como equipe, queremos retornar para a versão anterior se a nova versão apresentar problema.	**P1**	O rollback retorna o Deployment para `1.0.0`; o rollout conclui; a aplicação volta a responder; a equipe registra a evidência do procedimento.	PB-17
19	**PB-19**	**Documentação operacional**	Como equipe, queremos um roteiro de implantação e recuperação para repetir a demonstração e operar o sistema com segurança.	**P2**	README explica pré-requisitos, deploy, acesso, escala, atualização e rollback; comandos usam nomes reais do projeto; segredos não aparecem na documentação.	PB-10, PB-18
20	**PB-20**	**Evidências da entrega**	Como equipe, queremos registrar as evidências das atividades para comprovar a implantação e a operação do sistema.	**P2**	Repositório possui imagem única, manifests e workflows; existem evidências de Pods, Service, escalabilidade, self-healing, atualização e rollback; a demonstração prática pode ser repetida.	PB-10, PB-12, PB-14, PB-17, PB-18

Definition of Ready

Um item pode entrar em sprint quando:

- possui objetivo e critério de aceite compreensível;
- dependências anteriores foram resolvidas;
- equipe sabe quais arquivos, imagens ou recursos Kubernetes serão afetados;
- a forma de validar o resultado está definida.

Definition of Done

Um item será considerado concluído quando:

- a implementação estiver versionada no GitHub;
- os testes aplicáveis tiverem passado;
- a documentação e a configuração necessárias tiverem sido atualizadas;
- a equipe tiver validado o critério de aceite;
- a evidência da execução estiver registrada quando o item envolver Docker, GHCR ou Kubernetes.

Sequência recomendada para as sprints

1. **Sprint de base e implantação:** PB-01 a PB-10, iniciando obrigatoriamente pela unificação do frontend e da API em uma única imagem.
2. **Sprint de operação Kubernetes:** PB-11 a PB-14.
3. **Sprint de entrega e recuperação:** PB-15 a PB-20.

Essa sequência respeita as dependências e permite demonstrar primeiro um sistema funcional, depois sua capacidade de operação e por fim a atualização segura de versões.